

< Previous







Next >

Beyond standard binary coding

 Bookmark this page

Problem: Can numbers be simpler than the length of their binary representation?

Integers are central to an algorithmic information approach to artificial intelligence.

integer complexity

number	binary representation	length
1	1	1
2	10	2
3	11	2
4	100	3
5	101	3
6	110	3
...	...	...
10	1010	4
11	1011	4

- Integers are used to retrieve objects by their rank in lists. Even seemingly complex objects such as convoluted emojis are no more complex than their address in the set of emojis. And this address is nothing but an integer.
- Integers are much more important than that. Integers are involved in the formal description of most objects AI is dealing with.
- Integers are objects of interest in themselves as well, because of their role in mathematics (note that "real" numbers with limited precision are, in fact, pairs of integers).

So let's find a way to measure the complexity of integers.

Mathematics (since Leibniz and, before him, ancient Chinese scientists) and Computer science like to represent numbers with bits. A first estimate of complexity consists of counting the number of bits in the standard binary representation (see table).

One can see that with this method, the complexity of n , as measured by the length of its binary representation, is (for $n > 0$):

$\lceil \log_2(1+n) \rceil$

where $\lceil x \rceil$ (with upper brackets) designates the smallest integer larger than x .

Decimal to binary converter:

1234

Convert

$(1234)_{10} = (10011010010)_2$

This method, however, totally ignores the existence of "round" numbers. This is unfortunate. A round number like one billion (1 000 000 000 in base 10) looks much simpler than the 30 bits of its binary representation (111011100110101100101000000000).

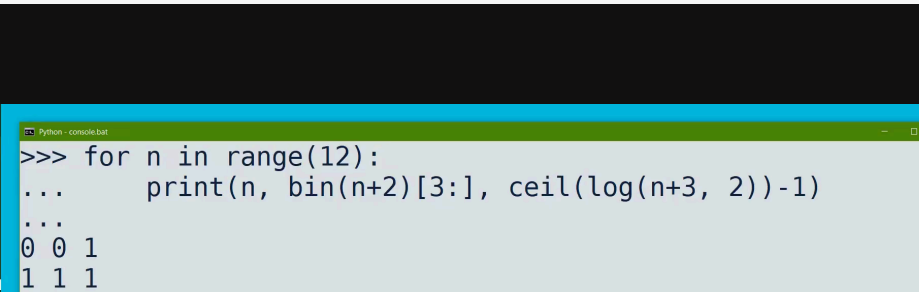
Binary to decimal converter:

10010110

Convert

$(10010110)_2 = (150)_{10}$

Integer complexity



```
>>> for n in range(12):
...     print(n, bin(n+2)[3:], ceil(log(n+3, 2))-1)
...
0 0 1
1 1 1
2 10 2
3 11 2
4 100 3
5 101 3
6 110 3
7 111 3
8 1000 4
9 1001 4
10 1010 4
11 1011 4
```

We subtract one because we get rid of the systematic leading 1.

This table showing our compact code can be generated using the

2 00 2
3 01 2
4 10 2
5 11 2
6 000 3
7 001 3
8 010 3
9 011 3
10 100 3
11 101 3
>>>

▶ 4:10 / 7:05

▶ 2.0x

can be generated using the following python lines:
To summarize our coding method,
we represent an integer n by considering the standard binary code of $n+2$, from which we drop the leading 1.
Our compact code is shorter by one bit

► Complexity of Integers in a nutshell

Question: Why are we ignoring space separators when counting the length of round numbers compact codes?

Answer: Separators are not symbols. You can't have two separators in a row. They may result from physical signal processing, as for the Morse code, or from low-level word segmentation (e.g. lists of strings in Python: '0010 11010 111' ↔ ['0010', '11010', '111']).
(Note: In Chapter 3, we will consider the possibility of eliminating all space separators altogether. Efficient space-free codings of integers can be designed.)

The above video proposed a compact coding method for integers:

compact code for integers

To code n , we consider the standard binary code of $(n+2)$ and we drop the initial 1 (which is redundant).
We then reintroduce a separated heading bit: 1 for "normal" numbers, and 0 for "round" numbers.

Coding Integers (1)

1/1 point (ungraded)

Using the coding convention just presented, what would be a good coding for 500?

☐ 1 11110100

☐ 0 01 11

☒ 0 11 00

☐ 1 01 11

☐ 1 1101

☐ 1 11110110

✓

Answer
Correct:
Since 500 is a 'round' number, it is advantageous to use the alternative code, prefixed by 0. Then we indicate the code for 5, which is 11 (code for 5+2, which is 111, stripped of the leading 1). Finally, we indicate the number of zeros by 00, which is the code for 2. So, 0 11 00 is the correct answer. Note that since 111110100 is the standard binary representation of 500), an

alternative answer would have been `1 11110110` , where `11110110` is the binary code of 502 without the heading `1` . It is however longer than `0 11 00` .

Submit

You have used 1 of 1 attempt

 Answers are displayed within the problem

Coding integers (2)

1/1 point (ungraded)

Which number does the following code represent: `0 110 11` ?

- ☐ 5000000000000
- ☐ 500000
- ☒ 1200000
- ☐ 120000

✓

Answer
Correct:
The answer is 1200000. `0` = alternative coding for round numbers `110` = code for 12 `11` = code for 5 zeros

Submit

You have used 1 of 1 attempt

 Answers are displayed within the problem

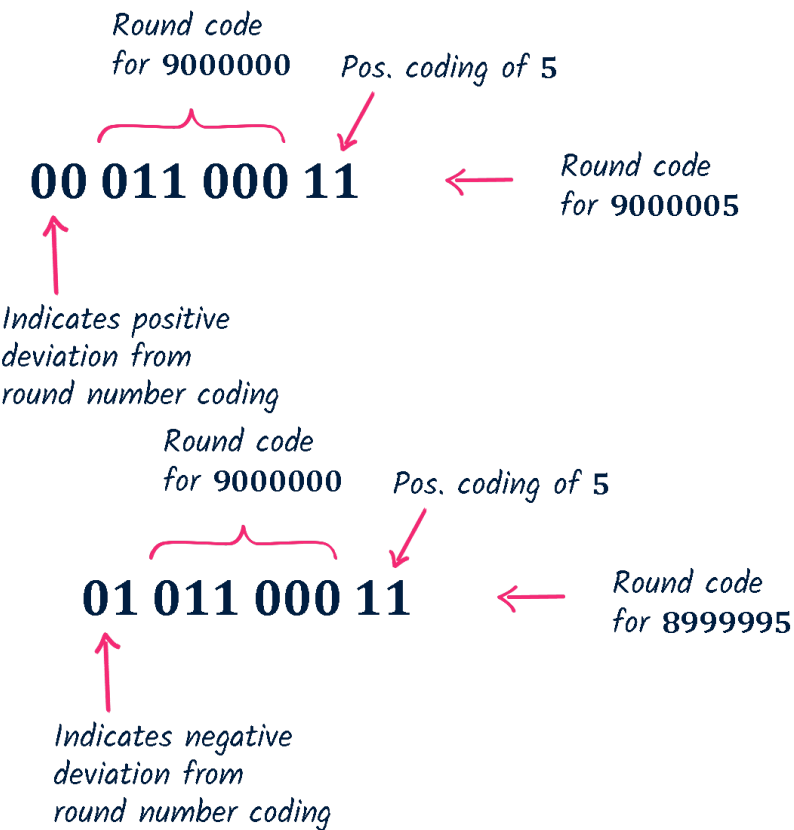
Our coding system acknowledges the fact that round numbers are simple. What about their immediate neighbours? Intuitively, `5000000011` isn't much more complex than `5000000000`, as it can be expressed as `5000000000` + `11`. The coding method illustrated in the figures below captures this idea. We prefix the code by `00` for a positive shift from the round number, and by `01` for a negative shift. Then we indicate the two-part code of the round number, and finally the compact code of the shift.

Decimal to binary converter:

Convert

Binary to decimal converter:

Convert



Caveat: These codes use space delimiters, as in the Morse code.

Complexity of 1000008

1/1 point (ungraded)

Based on the suggested coding, provide a short binary representation of 1000008?

00 1 000 010

✔ Answer: 00 1 000 010

Answer

Correct:

_ 1000008 is quite naturally described as 1000000 + 8.

If we use the code above, we get 00 1 000 010 , where 1 000 is the round code for 1000000 (= 1 + 6 zeroes) and 010 indicates the shift (8).

As a consequence, the complexity of 1000008 using this code is its length: 9 bits.

__ 1000008 is quite naturally described as 1000000 + 8. If we use the code above, we get 00 1 000 010 , where 1 000 is the round code for 1000000 (= 1 + 6 zeroes) and 010 indicates the shift (8). As a consequence, the complexity of 1000008 using this code is its length: 9 bits.

Submit

You have used 1 of 1 attempt

< Previous

Next >



edX

- About
- Affiliates
- edX for Business
- Open edX
- Careers
- News

Legal

- Terms of Service & Honor Code
- Privacy Policy
- Accessibility Policy
- Trademark Policy
- Sitemap
- Cookie Policy
- Your Privacy Choices

Connect

- Idea Hub

[Contact Us](#)
[Help Center](#)
[Security](#)
[Media Kit](#)



© 2025 edX LLC. All rights reserved.
深圳市恒宇博科技有限公司 [粤ICP备17044299号-2](#)